

INSTITUTO TECNOLÓGICO SUPERIOR DEL ORIENTE DEL ESTADO DE HIDALGO.

Programa Educativo:

Ingeniería en Sistemas Computacionales

Docente:

Efren Rolando Romero Leon

Materia:

Métodos Numéricos

Semestre: 4 Grupo: 2

Actividad:

Problemario

Alumnos:

Jeshua Avelar Martinez

Omar David Hernández Herrera

Fecha:

4 de Febrero de 2026

Códigos con errores

- 1.- Error de Redondeo Binario
- 2.- Pérdida de Precisión por Magnitud(IEEE 754)
- 3.- Comparación Directa con ==
- 4.- Acumulación de Errores en Bucles
- 5.- Cancelación por Resta (Loss of Significance)
- 6.- Desbordamiento Silencioso (Overflow)
- 7.- Conversión Estrecha (Narrowing Primitive Conversión)

Tipos de Errores

- 1.-Error Absoluto
- 2.-Error Relativo
- 3.-Error Porcentual

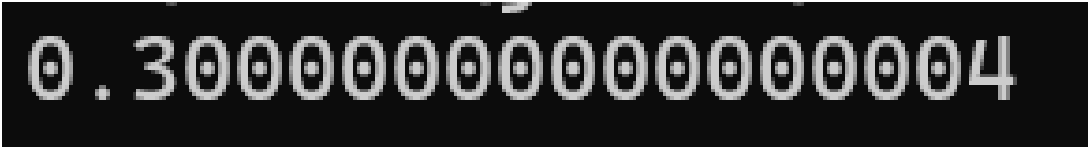
Ejemplos

- Problema 1: Balanza de la frutería
- Problema 2: Carpintería de precisión
- Problema 3: Control de temperatura
- Problema 4: Topografía en el atletismo
- Problema 5: Envasado de Líquido

Códigos con errores

1.- Error de Redondeo Binario

```
#include <iostream>
using namespace std;
int main() {
    cout << 0.1 + 0.2 << endl;
    return 0;
}
```



0.30000000000000004

Este error ocurre porque las computadoras representan los números en **binario**, no en decimal, y muchos valores decimales como **0.1** o **0.2** no pueden expresarse exactamente en base 2. Como resultado, estos números se almacenan como **aproximaciones** y no como valores exactos. Cuando se realizan operaciones aritméticas, estas pequeñas aproximaciones se combinan y el resultado final puede diferir ligeramente del valor matemático esperado, haciendo visible el error.

2.- Pérdida de Precisión por Magnitud(IEEE 754)

```
#include <iostream>
#include <iomanip>

int main() {
```

```

// Un double tiene aproximadamente 15-17 dígitos significativos
double numeroGrande = 1.0e16; // 1 seguido de 16 ceros
double numeroPequeno = 1.0;

double resultado = numeroGrande + numeroPequeno;

std::cout << "--- Demostración de Pérdida de Precisión ---" <<
std::endl;
std::cout << std::fixed << std::setprecision(0);
std::cout << "Número Grande:  " << numeroGrande << std::endl;
std::cout << "Número Pequeño:  " << numeroPequeno << std::endl;
std::cout << "Suma Resultante: " << resultado << std::endl;

// Verificación lógica
if (resultado == numeroGrande) {
    std::cout << "\nRESULTADO: El número pequeño 'desapareció'." <<
std::endl;
    std::cout << "La suma es igual al número original debido a la
falta de bits en la mantisa." << std::endl;
}

return 0;
}

```

```

--- Demostración de Pérdida de Precisión ---
Número Grande:  1.0E16
Número Pequeño:  1.0
Suma Resultante: 1.0E16

RESULTADO: El número pequeño 'desapareció'.
La suma es igual al número original debido a la falta de bits en la mantisa.

--- Solución con BigDecimal ---
Suma Exacta:      10000000000000001.0

```

La pérdida de precisión ocurre cuando se realizan operaciones entre números de magnitudes muy diferentes usando tipos como `double`, que tienen un número limitado de bits para representar cifras significativas. Cuando un número es muy grande, ocupa toda la precisión disponible y los cambios pequeños, como sumar `1`, ya no pueden representarse, por lo que se descartan. Esto provoca que el número pequeño “desaparezca” y el resultado sea idéntico al número grande original.

3.- Comparación Directa con ==

```

#include <iostream>
#include <cmath>
#include <iomanip>

```

```

int main() {
    // 1. El Problema: Error de redondeo en punto flotante
    double a = 0.1 + 0.1 + 0.1;
    double b = 0.3;

    std::cout << std::setprecision(17); // Mostrar toda la precision
    std::cout << "Valor de a (0.1 * 3): " << a << std::endl;
    std::cout << "Valor de b:          " << b << std::endl;

    // 2. Comparación directa (FALLA)
    std::cout << "\n--- Comparación con '==' ---" << std::endl;
    if (a == b) {
        std::cout << "Resultado: Son iguales" << std::endl;
    } else {
        std::cout << "Resultado: SON DIFERENTES (Error esperado)" <<
std::endl;
    }

    // 3. La Solución: Uso de epsilon
    double epsilon = 0.00001;

    std::cout << "\n--- Comparación con epsilon (" << epsilon << ")
---" << std::endl;
    if (std::abs(a - b) < epsilon) {
        std::cout << "Resultado: Son iguales (dentro del margen de
error)" << std::endl;
    } else {
        std::cout << "Resultado: Son diferentes" << std::endl;
    }

    return 0;
}

```

```

Valor de a (0.1 * 3): 0.30000000000000004
Valor de b:          0.3

```

```
--- Comparación con '==' ---
```

```
Resultado: SON DIFERENTES (Error esperado)
```

```
--- Comparación con Épsilon (1.0E-5) ---
```

```
Resultado: Son iguales (dentro del margen de error)
```

Este error se produce al comparar números de punto flotante usando el operador `==`, ya que este compara los valores **bit a bit**. Debido a los errores de representación y redondeo, dos resultados que matemáticamente deberían ser iguales suelen diferir en sus últimos bits. Aunque al imprimirlos parecen iguales, internamente no lo son, lo que provoca que la comparación directa falle y arroje resultados incorrectos.

4.- Acumulación de Errores en Bucles

```
#include <iostream>
#include <iomanip>
#include <cmath>

int main() {
    int iteraciones = 1000000; // Un millón de sumas
    double incremento = 0.1;

    // 1. Acumulación usando 'double'
    double sumaDouble = 0.0;
    for (int i = 0; i < iteraciones; i++) {
        sumaDouble += incremento;
    }

    // Resultado esperado
    double esperado = iteraciones * incremento;

    std::cout << std::setprecision(17);
    std::cout << "--- Acumulación en Bucle (1,000,000 de iteraciones)
---" << std::endl;
    std::cout << "Resultado esperado: " << esperado << std::endl;
    std::cout << "Resultado double:   " << sumaDouble << std::endl;
    std::cout << "Diferencia (Error): " << (sumaDouble - esperado) <<
std::endl;

    // 3. Verificación del error
    if (sumaDouble != esperado) {
        std::cout << "\nNOTA: El error de 'double' es notable tras un
millón de sumas." << std::endl;
    }

    return 0;
}
```

```

--- Acumulación en Bucle (1,000,000 de iteraciones) ---
Resultado esperado: 100000.0
Resultado double: 100000.00000133288
Diferencia (Error): 1.3328826753422618E-6

--- Solución con BigDecimal ---
Resultado exacto: 100000.0

NOTA: El error de 'double' es notable tras un millón de sumas.

```

La acumulación de errores sucede cuando una operación con punto flotante se repite muchas veces, como sumar **0.1** en un bucle. Cada operación introduce un pequeño error de redondeo, y al repetirse miles o millones de veces, estos errores se van acumulando progresivamente. Al final del proceso, la suma total presenta una desviación apreciable respecto al valor esperado, aun cuando cada error individual sea muy pequeño.

5.- Cancelación por Resta (Loss of Significance)

```

#include <iostream>
#include <iomanip>

int main() {
    // Dos números muy grandes y muy cercanos
    double x = 1234567890.1234567;
    double y = 1234567890.1234560;

    // El resultado esperado es 0.0000007
    double resultado = x - y;

    std::cout << std::setprecision(17);
    std::cout << "x = " << x << std::endl;
    std::cout << "y = " << y << std::endl;
    std::cout << "Resultado real: " << resultado << std::endl;

    return 0;
}

```

Resultado real: 7.152557373046875E-7

Este error ocurre al restar dos números muy grandes que son casi iguales entre sí. Al almacenarse en punto flotante, los valores pierden precisión en sus últimos dígitos debido a la limitación de la mantisa. Cuando se realiza la resta, los dígitos comunes se cancelan y el resultado depende únicamente de los dígitos menos precisos, produciendo un valor muy impreciso o incluso cero, aunque matemáticamente debería existir una diferencia pequeña.

6.- Desbordamiento Silencioso (Overflow)

```
#include <iostream>
#include <limits>

int main() {
    int max = std::numeric_limits<int>::max(); // 2,147,483,647
    int resultado = max + 1; // Desbordamiento

    std::cout << "Maximo int: " << max << std::endl;
    std::cout << "Maximo + 1: " << resultado << std::endl;

    // Detección manual del desbordamiento (estilo didáctico)
    if (resultado < max) {
        std::cout << "Error detectado: ¡Desbordamiento!" << std::endl;
    }

    return 0;
}
```

```
Máximo int: 2147483647
Máximo + 1: -2147483648
Error detectado: ¡Desbordamiento!
```

El desbordamiento de enteros se produce cuando una operación aritmética genera un resultado que excede el rango máximo que puede representar un tipo entero. Debido a que los enteros usan un número fijo de bits, al superar su límite el valor se “envuelve” y produce un resultado incorrecto, como pasar del máximo positivo al mínimo negativo. Este comportamiento depende del lenguaje y del hardware, pero siempre se debe a la **limitación del tamaño de memoria** asignado al tipo de dato.

7.- Conversión Estrecha (Narrowing Primitive Conversión)

```
#include <iostream>
#include <iomanip>
#include <limits>

int main() {
    double valorDouble = 3.14159e10; // Número muy grande
    int valorInt = static_cast<int>(valorDouble); // Casting explícito
```



```
std::cout << std::setprecision(15);
std::cout << "Valor Original: " << valorDouble << std::endl;
std::cout << "Valor Truncado: " << valorInt << std::endl;

return 0;
}
```

```
Valor Original: 3.14159E10
Valor Truncado: 2147483647
```

Este error ocurre cuando se convierte un valor de un tipo con mayor rango o precisión, como `double`, a un tipo más limitado, como `int`. Si el valor original no cabe dentro del rango del tipo destino, se pierde información durante la conversión, lo que puede provocar truncamiento, saturación o resultados inesperados. La causa principal es asumir que todos los valores pueden convertirse de forma segura sin verificar previamente los límites del tipo destino.

Tipos de Errores

Estos errores son herramientas importantes en el análisis numérico, ya que nos permiten evaluar y comunicar la precisión de nuestros cálculos. Nos ayudan a comprender qué tan cerca estamos del valor verdadero y cuánta confianza podemos tener en nuestros resultados.

1.-Error Absoluto

¿Qué es?

El error absoluto es una medida de la diferencia entre el valor verdadero y el valor aproximado de un cálculo numérico. Representa la magnitud del desvío o la imprecisión en el resultado obtenido.

¿Cómo se calcula?

Se calcula restando el valor verdadero al valor aproximado. El error absoluto puede ser positivo o negativo, dependiendo de si el valor aproximado está por encima o por debajo del valor verdadero.

Fórmula

$$E_a = |V_r - V_m|$$

2.-Error Relativo

¿Qué es?

El error relativo es una medida relativa de la precisión de un cálculo numérico. Se calcula dividiendo el error absoluto entre el valor verdadero y se expresa generalmente como un valor decimal o porcentual.

¿Cómo se calcula?

Se calcula dividiendo el error absoluto entre el valor verdadero y se expresa generalmente como un valor decimal o porcentual. El error relativo permite comparar la magnitud del error con respecto al tamaño del valor verdadero.

Fórmula

$$Er = Ea/Vr$$

3.-Error Porcentual

¿Qué es?

El error porcentual es una medida del error relativo expresado como un porcentaje. Es útil para evaluar y comunicar de manera más intuitiva la precisión de un cálculo numérico.

¿Cómo se calcula?

Se calcula dividiendo el error absoluto entre el valor verdadero y multiplicándolo por 100%. El error porcentual indica qué tan lejos está la aproximación del valor verdadero en términos porcentuales.

Fórmula

$$Ep = Er \times 100\%$$

Ejemplos

Problema 1: Balanza de la frutería

Un cliente compra una sandía cuyo peso real, según una báscula de alta precisión, es de 500g. Sin embargo, la balanza del supermercado marca 485g. Calcula los errores de la medición.

Absoluto: $Ea = |500 - 485| = 15g$

Relativo: $Er = 15/500 = 0.03$

Porcentual: $Ep = 0.03 \times 100 = 3\%$

Problema 2: Carpintería de precisión

Un carpintero necesita instalar una puerta que mide exactamente 2.00 m de altura. Al realizar el corte con su sierra manual, la medida final de la puerta resulta ser de 2.05 m. Determina el error absoluto, relativo y porcentual.

$$\text{Absoluto: } E_a = |2.00 - 2.05| = 0.05$$

$$\text{Relativo: } E_r = 0.05/2.00 = 0.025$$

$$\text{Porcentual: } E_p = 0.025 \times 100 = 2.5\%$$

Problema 3: Control de temperatura

Una receta de repostería exige que el horno esté a una temperatura real de 180 °C. No obstante, el termómetro interno del horno indica que la temperatura es de 171 °C. Calcula el margen de error porcentual de este termómetro.

$$\text{Absoluto: } E_a = |180 - 171| = 9^\circ\text{C}$$

$$\text{Relativo: } E_r = 9/180 = 0.05$$

$$\text{Porcentual: } E_p = 0.05 \times 100 = 5\%$$

Problema 4: Topografía en el atletismo

En una competencia de atletismo, se utiliza un telémetro láser para medir una pista de 100 m planos. El dispositivo arroja una lectura de 100.2 m. ¿Cuál es el error relativo de esta medición?

$$\text{Absoluto: } E_a = |100 - 100.2| = 0.2\text{m}$$

$$\text{Relativo: } E_r = 0.2/100 = 0.002$$

$$\text{Porcentual: } E_p = 0.002 \times 100 = 0.2\%$$

Problema 5: Envasado de Líquido

Una fábrica de jugos etiqueta sus envases indicando que contienen 1000 ml. Al realizar un control de calidad en la línea de producción, se descubre que una botella contiene realmente 990 ml. Encuentra los tres tipos de error.

$$\text{Absoluto: } E_a = |1000 - 990| = 10\text{ml}$$

$$\text{Relativo: } E_r = 10/1000 = 0.01$$

$$\text{Porcentual: } E_p = 0.01 \times 100 = 1\%$$